Ghana MoMo Smishing Detection

A Mobile-Money SMS Phishing Classifier for the Ghanaian Context

Technical Documentation & Methodology Report

Version 1.0
August 2026

Contents

1. Project Overview

This project builds a machine-learning system that classifies mobile-money (MoMo) SMS messages in the Ghanaian context into risk tiers, so that scam messages — smishing — can be flagged before they reach or mislead a user. The system is tailored to local conditions: it draws on real Telecel Cash and bank transaction message formats, covers Ghanaian telco brands (MTN, Telecel, AirtelTigo), and handles English–Twi code-switching.

The work spans the full applied-ML lifecycle: dataset assembly and cleaning, leakage-safe splitting, a preprocessing layer hardened against obfuscation, a baseline classifier, an adversarial audit of that classifier, and a three-tier risk model. A central outcome of the project is a rigorous negative finding about the training data, documented in Section 7, which is itself a meaningful research contribution.

1.1 Terminology

- **Smishing:** phishing carried out over SMS.
- **MoMo:** mobile money — wallet-based payments (MTN MoMo, Telecel Cash, AirtelTigo Money).
- **Obfuscation:** deliberate character-level tricks (look-alike letters from other alphabets, invisible characters) used to slip a scam past text filters.
- **Leakage:** when information from the test set is inadvertently available during training, producing inflated and untrustworthy scores.

2. Objectives & Scope

2.1 In Scope

- Classify SMS messages into three risk tiers: Safe, Suspicious, and Dangerous.
- Cover the locally relevant scam families: fake reversal / wrong-send refund requests, fake prize and promotional messages, and fake bank/telco alerts requesting a click or call.
- Preprocess messages so the classifier is robust to Unicode confusable and invisible-character evasion.
- Preserve user privacy by redacting personally identifiable information (PII) to placeholder tokens before any storage or training.

2.2 Out of Scope (Version 1)

- Real-time on-device or telecom-integrated deployment.
- Email phishing (the pipeline is SMS-only; email rows in source data are excluded).
- Languages beyond English and English–Twi code-switching.

3. Data Assets

Four source files were assembled during the project. The table below summarises each and its role.

File	Rows	Role
momo_sms_dataset_v2.jsonl	4,600	Primary training data — richest schema (mechanism tags, normalized text, obfuscation flags)
momo_sms_dataset_generated (1).jsonl	4,600	Predecessor set — supplementary, deduplicated against v2 before use
ghana_momo_phishing_dataset_synthetic_1000.csv	1,000	Mixed SMS/email; only the 457 SMS rows are used

File	Rows	Role
obfuscation_validation_set.jsonl	13	Held-out probe for the obfuscation normalizer — never used for training

3.1 Schema (Unified)

All sources are reconciled into one internal table with a consistent schema:

Column	Description
text	The message as displayed (PII redacted to placeholders)
normalized_text	Message after NFKC normalization, confusable folding, and invisible-character stripping
label	Binary target: 0 = legitimate, 1 = phishing/scam
category	Scam family or 'legitimate' (e.g. reversal, promo, alert, banking, impersonation)
mechanism	Cognitive-exploitation tag where known (authority, fear, greed, scarcity, urgency, etc.)
source_file	Provenance tag (v2 / generated_v1 / csv)

3.2 Privacy Handling

All personally identifiable information is redacted to placeholder tokens — <PHONE>, <NAME>, <AMOUNT>, <ACCOUNT>, <REF>, <URL> — before any message is stored or used for training. Legitimate templates are derived from real transaction messages with these redactions applied, so the model learns genuine message shape without retaining private data.

4. Preprocessing & Obfuscation Defence

The module normalize.py runs before the model sees any message. Its design principle is to teach the model that something was hidden, rather than to memorise specific tricks — this generalises to evasion techniques not seen in training.

- **NFKC normalization** — canonicalises Unicode form.
- **Confusable folding** — maps look-alike Cyrillic, Greek, and full-width characters to their Latin equivalents.
- **Invisible-character stripping** — removes zero-width and other non-printing characters.
- **Detection flags** — boolean features (had_confusable, had_invisible_char, obfuscation_suspected) that fire on any obfuscation attempt and are fed to the model alongside the normalized text.

Validation: the 13-case held-out obfuscation set is used to confirm the normalizer catches confusable and zero-width tricks it was not tuned on.

5. Pipeline & Methodology

The system is built as an ordered pipeline; each stage produces an artifact consumed by the next.

5.1 Stage 1 — Unify (unify.py)

Reconciles the four source schemas into one table (unified_sms.csv, 9,650 rows). Email rows from the CSV are dropped; the predecessor JSONL is deduplicated against v2 on message text; the obfuscation set is deliberately left untouched.

5.2 Stage 2 — Leakage-safe Split (dedup_and_split.py)

Because scam messages are template-generated, many are near-duplicates differing only in amounts, phone numbers, or dates. A 'skeleton' is derived for each message by masking those variable details; all messages sharing a skeleton are forced into the same split. This prevents a template appearing in both training and test — the most common cause of inflated scores.

Result: 5,790 train / 1,930 validation / 1,930 test, with identical 53.8% scam balance across all three splits and zero skeleton overlap between splits (verified).

5.3 Stage 3 — Baseline (baseline.py)

A deliberately simple, fast, interpretable model establishes how hard the problem is before heavier methods are considered:

- TF-IDF features: word 1–2 grams plus character 3–5 grams (the character grams catch code-switched Twi tokens and obfuscation residue).
- Classifiers: Logistic Regression and Linear SVM, with balanced class weights.
- Decision threshold tuned on validation, never on test.

5.4 Stage 4 — Adversarial Audit (stress_test.py)

A robustness check that neutralises the most class-discriminative tokens in the test set and re-evaluates, measuring how much of the score depends on surface giveaways rather than genuine signal. This stage produced the project's key finding (Section 7).

5.5 Stage 5 — Three-Tier Risk Model (triage_model.py)

Converts the two-class problem into the requested three tiers using calibrated confidence rather than inventing a third class from nothing:

- A calibrated classifier outputs an honest probability that a message is a scam.
- Two thresholds, chosen on validation to meet a stated policy, cut that probability into Safe / Suspicious / Dangerous bands.
- A documented severity override pushes high-confidence scams in money-movement or credential categories (reversal, banking, impersonation) firmly into Dangerous.

6. Results

6.1 Baseline Performance

On the held-out test set, the TF-IDF + Logistic Regression baseline reached near-perfect scores (F1 ≈ 1.00). As explained in Section 7, this is a symptom of the data rather than evidence of a strong detector.

6.2 Three-Tier Assignment (Test Set)

Band	Legitimate	Scam	Interpretation
Safe	855	0	No scams misrouted as safe
Suspicious	16	0	Near-empty — see Section 7
Dangerous	21	1,038	98% precision on flagged messages

As a triage design the structure is sound: zero scams leaked into the Safe band, and the Dangerous band is 98% true scams. The Suspicious band is nearly empty because the model is rarely uncertain on this data — a direct consequence of the finding below.

7. Key Finding: Synthetic-Data Separability

The most important result of the project is a negative one, and it is documented honestly here because it determines what the current model can and cannot claim.

Observation. The baseline achieves ~100% F1, and this score is near-invariant when the most class-discriminative tokens are neutralised (accuracy fell only from 1.000 to 0.999).

Diagnosis. The synthetic generator produced two stylistically separable distributions. Legitimate messages systematically contain words such as 'balance' and 'confirmed' and explicit dates; scam messages systematically do not. The classifier can therefore separate the classes by surface style alone, without learning anything about manipulation.

Implication. The benchmark does not measure real smishing-detection ability. A high score here would not transfer to real inboxes, where legitimate promos also contain links and urgency, and scams also quote balances and dates.

This is a standard and valuable form of applied-ML self-audit: a model can score perfectly and still be unfit for purpose if the data permits a shortcut. Identifying this is a genuine contribution, not a failure.

8. Application Integration (Pilot)

The trained model is packaged for integration into an SMS fraud-detection application. Because the model has not yet been validated on real-world messages (Section 7), this integration is specified as a pilot / shadow-mode deployment: the app runs the model and surfaces its output as advisory guidance, while real messages are collected to validate performance before the model is relied upon in production.

8.1 Inference Interface

The module predict.py exposes a single class, TriageClassifier, that wraps the full inference path. Critically, it normalises every incoming message with normalize.py before classifying — the model was trained on normalised text, so raw input must pass through the same preprocessing or predictions are invalid.

```
from predict import TriageClassifier

clf = TriageClassifier("triage_model.joblib")
result = clf.classify("Your MoMo wallet is blocked. Verify PIN: bit.ly/x")

# result -> {
#   "band": "dangerous",
#   "scam_probability": 0.99,
#   "obfuscation_suspected": false,
#   "reasons": ["high scam probability (0.99)"],
#   "advisory_only": true
# }
```

8.2 Message Processing Flow

Each message passes through the following stages inside the app:

- Ingest the raw SMS text (with the user's consent to scan).
- Normalise via normalize.py — folds look-alike characters, strips invisible characters, and raises obfuscation flags.
- Vectorise with the fitted TF-IDF vectorisers and score with the calibrated classifier to obtain a scam probability.
- Map the probability to a band (Safe / Suspicious / Dangerous), applying the category severity override where applicable.

- Return the band plus human-readable reasons; the app presents this as advisory guidance, not a verdict.

8.3 Recommended App Behaviour by Band

Band	Suggested app action
Safe	No interruption; message shown normally.
Suspicious	Show a soft caution banner; invite the user to review or report.
Dangerous	Prominent warning; advise against clicking links, calling numbers, sharing a PIN, or sending money; offer a one-tap report.

In all cases the app should make clear that the assessment is automated and advisory. **The model must never auto-delete, auto-block, or take irreversible action on a message during the pilot.**

8.4 Shadow-Mode Validation Loop

The pilot is designed to close the data gap identified in Section 7. While running, the app should:

- Log each message's band and probability alongside (with consent) the anonymised, PII-redacted message, so a real-world evaluation set accumulates naturally.
- Provide a user 'report / correct' control, turning user feedback into ground-truth labels.
- Periodically compare model bands against user-confirmed outcomes to measure real precision and recall — the numbers that actually matter.

8.5 Pre-Production Checklist

The following must be satisfied before the model graduates from pilot to a relied-upon production detector:

- A real-world evaluation set (target: at least a few hundred genuine Ghanaian scam and legitimate SMS) is assembled and the model is tested on it.
- Real-world phishing recall and precision meet an agreed threshold — in particular, the false-negative rate (dangerous messages labelled safe) is acceptably low.
- The probability distribution shows a populated Suspicious band on real data, confirming the model expresses genuine uncertainty.
- Privacy, consent, and data-retention terms for scanned messages are reviewed and documented.
- A rollback and monitoring plan is in place for post-deployment drift.

Until these are met, the integration remains a pilot and its outputs are advisory. This staged approach is standard practice for deploying an ML model whose real-world performance is not yet established.

9. Limitations

- The training data is synthetic and trivially separable; reported scores overstate real-world performance.
- Model confidence is poorly spread, so the Suspicious tier is under-populated on current data.
- The category column mixes two labeling schemes (v2's 4 categories and the CSV's 12 scam types); a unified taxonomy is future work.
- mechanism tags exist only for the v2 subset, limiting mechanism-level analysis.
- No real-world evaluation set yet exists, so external validity is unverified.

10. Recommended Next Steps

In priority order:

- **Collect a small real-world evaluation set.** Even 100–200 real Ghanaian scam and legitimate SMS (crowdsourced, or via the Cyber Security Authority's reporting channel) would provide a benchmark that actually measures skill. Train on synthetic, test on real.
- **Redesign the generator** so both classes share vocabulary, register, and structure — making psychological manipulation the only separating signal. The stress test becomes the acceptance criterion: a good generator is one the baseline cannot trivially solve.
- **Adversarially audit every future dataset** with the token-neutralisation check before trusting any score.
- **Advance to a transformer (XLM-RoBERTa)** only once the data supports it; on trivially separable data it offers no advantage over the linear baseline.

11. Repository Artifacts

Artifact	Description
unify.py	Reconciles four sources into unified_sms.csv
dedup_and_split.py	Skeleton-based, leakage-safe train/val/test split
baseline.py	TF-IDF + Logistic Regression / Linear SVM baseline
stress_test.py	Adversarial token-neutralisation audit
triage_model.py	Calibrated three-tier (Safe/Suspicious/Dangerous) model
predict.py	Inference wrapper — classifies a raw SMS into a risk band (app integration)
normalize.py	Preprocessing + obfuscation-detection module
triage_model.joblib	Saved vectorisers, calibrated classifier, thresholds, severity map
unified_sms.csv / train / val / test	Unified dataset and its splits

End of document.